



SNAKE

MIKRODATORPROJEKT TSIU51, GRUPP 8

FILIP LINDBERG, ERIK PETTERSSON, AHMAD SOLTANI

Innehåll

1 Inledning.....	3
1.1 Allmänt.....	3
1.2 Bakgrund	3
1.3 Mål.....	3
1.4 Kravspecifikation.....	4
2 Översikt.....	5
2.1 Arbetsuppdelning.....	5
2.2 Komponenter	5
2.3 Blockschemata.....	5
3 Projektet	6
3.1 Processor ATmega328p.....	6
3.1.1 I/O (Inputs/Outputs).....	6
3.2 Display Matrix (8x8)	7
3.2.1 SPI protokollen.....	7
3.3 Joystick.....	8
3.3.1 Joystick struktur.....	9
3.4 Rörelse (FIFO).....	10
3.4.1 Videominnet (SNAKE_FIFO).....	10
3.4.2 Översättning av koordinater (SET_PIX).....	11
3.4.3 Push och Pop.....	11
3.4.4 FIFO JSP-Diagram.....	12
3.5 Muxen.....	13
4 Diskussion	15
4.1 Misstag	15
4.2 Slutsats.....	15
5 Figurbeskrivning.....	16
6 Referenslista	17

1 Figurförteckning

Figur 1: Blockschemata.....	5
Figur 2: De I/O pinar som inkluderar de 24 pinnarna för Port B, Port C och Port D och även 4 andra enskilda I/O pinnar	6
Figur 3: SPI Master-Slave Interconnection	7
Figur 4: Kodexempel på pinndefinitioner och HW_INIT för start av SPI.....	8
Figur 5: Endast de två högsta bitarna från X respektive Y-led från joystickens är relevanta för projektet och används för att läsa av joystickens position. Spelaren kan enbart gå vågrätt eller lodrätt, vilket gör att det endast är de gråmarkerade rutorna i diagrammet	8
Figur 6: Kodutdrag för joystick funktionen. ADMUX sätts upp för att hantera Y-axel respektive X-axel, omvandling startas och joystickvärde söks. Vid hittat värde lagras loopcount värde.....	9
Figur 7: Initiering av interrupten för joystick	9
Figur 8: Minnes deklarationen.....	10
Figur 9: Rörelselooopen i FIFO:n	10
Figur 10: Illustrering av minneshantering	10
Figur 11: Kodexempel på FIFO_PUSH och FIFO_POP.....	11

Figur 12: FIFO JSP-diagram	12
Figur 13: Kortfattat JSP schema över muxrutinen	13
Figur 14: Main rutinen för MUX.....	13
Figur 15: Koden för att skicka VMEM till skärmen. RAD_INDEX incrementas mellan varje MUX anrop	14
Figur 16: Rutinen för att uppdatera VMEM med ny information från till exempel rörelse rutinen	14

2 Inledning

2.1 Allmänt

Denna rapport täcker processens gång för projektet Snake. Snake är ett spel gjort i programmeringsspråket assembler, som körs på en ATmega328P processor. Genom att läsa denna rapport kommer läsaren att få en grundlig och omfattande förståelse för hur kommunikationen mellan hårdvara och mjukvara fungerar. Rapporten innehåller även en analys av de utmaningar och problem som uppstod under projektet, samt de lösningar som implementerades för att övervinna dem. Genom att läsa denna rapport kommer läsaren att få en djupare förståelse för datateknik och hur den kan tillämpas i praktiken.

2.2 Bakgrund

Spelet Snake är ett klassiskt spel som har funnits i olika former sedan slutet av 1970-talet. Spelet går ut på att styra en virtuell Snake, som rör sig över spelplanen och samlar mat. Maten är utspridd över planen. Ju mer mat Snaket äter, desto längre blir den, vilket ökar svårighetsgraden genom att öka risken för att Snaken kolliderar med sin kropp eller går ut ur planen. Vilket i sin tur leder till "Game Over", och spelet avslutas. Spelets popularitet har lett till många varianter och utvecklingar på olika plattformar och i olika programmeringsspråk.

2.3 Mål

Syftet med detta projekt är att ge deltagarna en fördjupad förståelse för datateknik och dess praktiska tillämpningar. Projektet fokuserar på att bekanta deltagarna med mikroprocessorbaserade hårdvaror och ge möjligheter för att skapa ett mindre datorsystem med hjälp av mikrokontroller och kring enheter. För att göra detta krävs kunskaper inom binär aritmetik och logik, vilket deltagarna kommer att lära sig under projektets gång.

Utöver detta är projektet också avsett att förbättra deltagarnas förmåga att samarbeta, kommunicera och felsöka problemen inom en mindre grupp. Genom att arbeta tillsammans och lösa tekniska problem kommer deltagarna att utveckla sina färdigheter och bli mer effektiva och självständiga inom datateknik.

2.4 Kravspecifikation

Projektet inleddes med att diskutera de ”skall-kraven” som projektet behövde uppfylla. Dessa krav togs fram med hjälp av kunden och blev hårda krav för att projekt skulle anses som godkänt och fungerande. Utöver de satta ”skall-kraven” så diskuterades även potentiella utökade krav för att bygga vidare på de grundläggande funktionerna. Nedan finns de kraven som har kommits överens mellan gruppen och kund.

Kravspecifikation
Beskrivning: Apparaten skall fungera som en spelkonsol till en tolkning av det klassiska spelet Snake. En 8x8 DAmatrix används som display. Spelaren styr en orm och äter frukter (visas som punkter) för att dess svans skall växa. Om spelaren nuddar en vägg avslutas spelet.
Skall-krav: 1. En spelare skall kunna styra ormen med en joystick 2. ”Slumpmässiga” objekt skall dyka upp på displayen 3. När ormen äter ett objekt skall ormens svans växa 4. Om ormen åker in i en vägg eller om den åker in i sin svans avslutas spelet 5. Ormen och objekten skall visas i olika färger
Utökade krav: 1. 2-Spelare som kör mot varandra 2. Meny till ett- eller tvåspelarläge eller spelinställningar 3. Slumpmässiga hinder som spelaren inte får åka in i 4. Olika typer av frukter som ger olika förmåner 5. Ljud när en frukt äts eller spelaren förlorar 6.- ”Booster” för att kunna åka snabbare framåt i en kort stund.

3 Översikt

Inledande text

3.1 Arbetsuppdelning

I början av projektet bestämdes vilka delar av projektet som behövde genomföras i grupp och vilka delar som kunde utföras individuellt av gruppmedlemmarna. Ett stort fokus låg på att få displayen fungera så fort som möjligt, vilket alla medlemmar i gruppen hjälptes åt med.

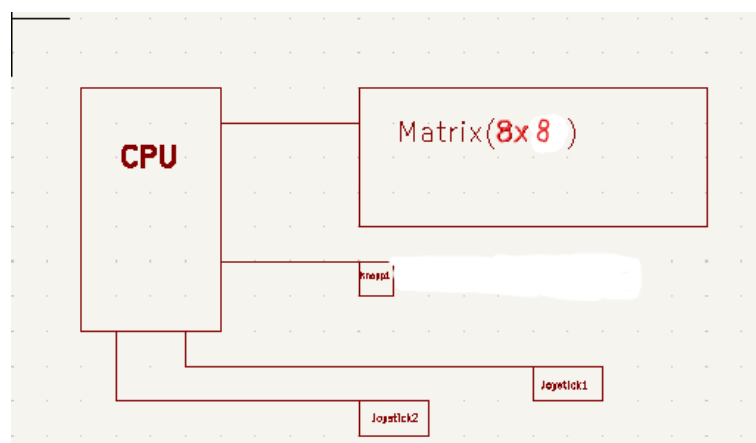
Resten av arbetet var ganska jämnt fördelat och det gjordes kontinuerliga diskussioner och samarbeten inom gruppen. I den inledande fasen av projektet kom gruppen fram till att använda fyra 8x8 DAmatrix displayer, för att få en display som var 16x16. Men efter många problem gjordes beslutet i samband med kund att minska till enbart en 8x8 DAmatrix, något som underlättade projektet signifikant.

3.2 Komponenter

Lista över Komponenter
• En DAmatrix (8x8) • En joystick • En knapp • En ATmega328P

3.3 Blockschema

Eftersom projektet har genomförts på ett färdigt monterat DAvid kort, ser blockschemat inte speciellt komplicerat ut. Joystick nummer 2 som är med i bilden nedan, har inte använts i projektet.



Figur 1: Blockschema.

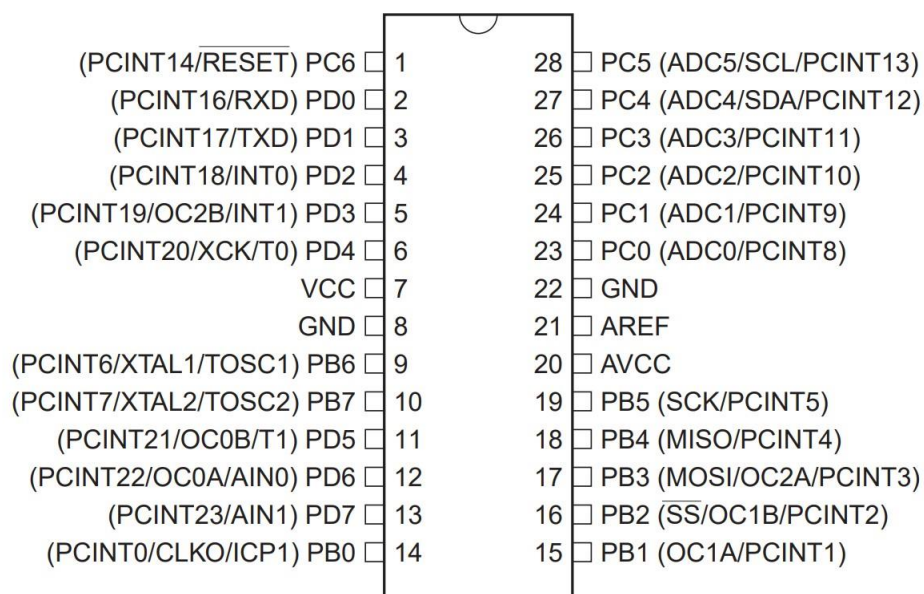
4 Projektet

4.1 Processor ATmega328P

Processorn ATmega328P är en populär 8-bitars mikrokontroller som hör till Atmel AVR (Advanced Virtual RISC). ATmega328P har på grund av sin låga strömförbrukning och höga hastighet en stor användning inom inbyggda system och elektroniska produkter, såsom robotar och hemautomationer. Processorn har ett Flash Memory på 32KB som används för att lagra Datat, och ett SRAM (Static Random Acces Memory) på 2KB som används för att lagra Datat tillfälligt. Dessutom har processorn en klockgenerator som kan ställas in på olika frekvenser upp till en maximal frekvens på 20MHz.

4.1.1 I/O (Inputs/Outputs)

ATmega328p är utformat för att hantera en mängd olika uppgifter genom sina 32 pinnar. Av dessa pinnar utgör 28 I/O pinnar som är



Figur 2: De I/O-pinnar som inkluderar de 24 pinnarna för Port B, Port C och Port D och även 4 andra enskilda I/O-pinnar

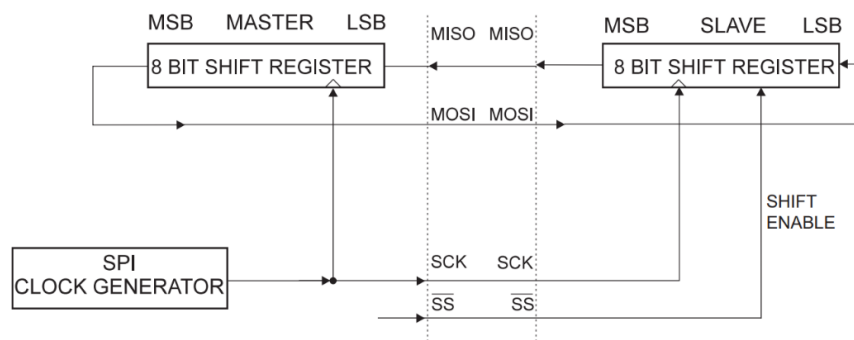
För mer information om de andra fyra pinnar som inte ingår i I/O-pinnarna och dess speciella funktioner kan man läsa på databladet. Det är även förklarat i detaljer om hur I/O-pinnarna sitter på chipet på databladet.

4.2 Display Matrix (8x8)

DAmatrix 8x8 är en display som består av 64 små lysdioder Varje lysdiod kan tändas eller släckas individuellt för att skapa olika mönster och tecken på skärmen. Displayen Matrix (8x8) används ofta i olika typer av elektroniska projekt, till exempel i digitala klockor, avancerade leksaker osv.

4.2.1 SPI protokollen

I detta projekt har SPI-protokollen använts för kommunikationen mellan ATmega328P processorn och displayen. SPI står för Serial Peripheral Interface, som betyder att Data skickas bit för bit i en serie, mellan Master-enheten (ATmega328P) och Slave-enheten (Matrix 8x8) i detta fall.



Figur 3: SPI Master-Slave Interconnection.

Interkommunikationen består av två skiftregister och en master-klockgenerator. Data skiftas alltid från Master till Slaven på MOSI-linjen (Master Out Slave In) och från Slaven till Mastern på MISO-linjen (Master In Slave Out).

För att starta kommunikation via SPI, dras Mastern SS (Slave Select) för den önskade Slave. Sedan efter varje dataöverföring synkroniserar Mastern Slaven genom att höja SS- linjen. I kodsnittet nedan visas en konfiguration av de hårdvaror som ingår SPI-kommunikationen.

```

1 .equ    SS = PB2           ; SS, pinnen (PB2) på processorns som är för Slave Select-signalen.
2 .equ    DD_SCK = PB5       ; Datainriktningen för klocksignalen på PB5
3 .equ    DD_MOSI = PB3      ; Datainriktning från master till slave på PB3
4 .equ    PORT_SPI = PORTB   ; Port B används för SPI
5 .equ    DD_OC1A = PB1
6
7 HW_INIT:
8 ; Set MOSI and SCK output, all others input
9     ldi    r16, (1<<DD_MOSI)|(1<<DD_SCK)|(1<<SS)|(1<<DD_OC1A)
10    out    DDRB, r16
11
12 ; Enable SPI, Master, set clock rate fck/64
13     ldi    r16, (1<<SPE)|(1<<MSTR)|(0<<SPR0)|(1<<SPR1) //DISPLAY
14     out    SPCR, r16
15     cbi    PORT_SPI, SS
16

```

Figur 4: Kodexempel på pinndefinitioner och HW_INIT för start av SPI

4.3 Joystick

Till projektet användes en joystick för att styra ormen. Joysticken ger en analog utspänning mellan 0 och 5 V beroende på position. Denna spänning omvandlas till en digital signal via ATmega328P:s A/D-omvandlare (Analog-till-Digital-omvandlare) där de två högsta bitarna av signalen avläses för att lokalisera joystickens position. Spelaren kan med hjälp av joysticken byta färdriktning på ormen för att undvika att krocka med väggar eller sig själv och röra sig mot nästa mål.

	Vänster		Höger	
Upp	11			
	10/01			
Ner	00			
	00	10/01	11	

Figur 5: Endast de två högsta bitarna från X- respektive Y-led från joystickens position. Spelaren kan enbart gå vågrätt eller lodrätt, vilket gör att det endast är de gråmarkerade rutorna i diagrammet

4.3.1 Joystickens struktur

Eftersom spelstrukturen gör att spelarens karaktär aldrig får stå stilla och bara kan röra sig rakt vågrätt eller lodrätt så är joystickens exakta läge inte alltid helt relevant. Exempelvis så ska ormen fortsätta röra sig i samma riktning även om joysticken åker tillbaka till mittläget.

På grund av detta så kan inte positionsvärdet från joysticken användas direkt, utan måste först filtreras till ett av de accepterade värdena, det vill säga, upp, ner, vänster, eller höger. Detta görs genom en loop som kollar de fyra olika accepterade värdena. Loopen kontrollerar ifall de två högsta bitarna från A/D-omvandlaren ger ”00” eller ”11” för först Y-axeln, följt av samma kontroll för X-axeln. Mellan varje avstämning så ökas registret r17 med ett. Detta värde sparas om jämförelsen i loopen stämmer, om inget accepterat värde hittas så förblir värdet det samma. Detta returnerar då ett värde mellan ett och fyra som då kan översättas till en rörelseriktning för vilket håll ormen ska röra sig.

```
ldi    r16, (0<<CS22)|(1<<CS21)|(1<<CS20) //JOYSTICK interrupt
sts    TCCR2B, r16
ldi    r16, (1<<TOIE2)
sts    TIMSK2, r16
```

Figur 7: Initiering av interrupten för joysticken.

```
CHANNEL:
Y_LED:
    lds    r16, ADMUX
    andi   r16, $fc
    ori    r16, $03
    rcall  CONVERT
X_LED:
    lds    r16, ADMUX
    andi   r16, $fc
    ori    r16, $02
    rcall  CONVERT
EXIT_JOY:
    ret

CONVERT:
    sts    ADMUX, r16
    lds    r16, ADCSRA
    ori    r16, (1<<ADSC)
    sts    ADCSRA, r16
WAIT:
    lds    r16, ADCSRA
    sbrc   r16, ADSC
    rjmp   WAIT
    lds    r16, ADCH
    inc    r17
    cpi    r16, $00
    breq   STORE
    inc    r17
    cpi    r16, $03
    breq   STORE
    jmp    DONE
STORE:
    sts    MOV_DIR, r17
DONE:
    ret
```

Figur 6: Kodutdrag för joystickfunktionen. ADMUX sätts upp för att hantera Y-axel respektive X-axel, omvandling startas och joystickvärde söks. Vid hittat värde lagras loopcountvärdet i r17-registret.

4.4 Rörelse (FIFO)

Ormens rörelse och position hanteras i ett 64 Bytes stort FIFO (First In First Out) minne med namn Snake FIFO. Rörelserutinen använder sig av joystickrutinens output för att välja vilket håll ormen ska röra sig och lagrar varje koordinat i ormens kropp i SNAKE_FIFO.

Utöver rörelse och position så har funktionen även rutiner för att hantera ”koordinat-översättning”, mer om detta i 3.4.2, samt kollisionskontroll.

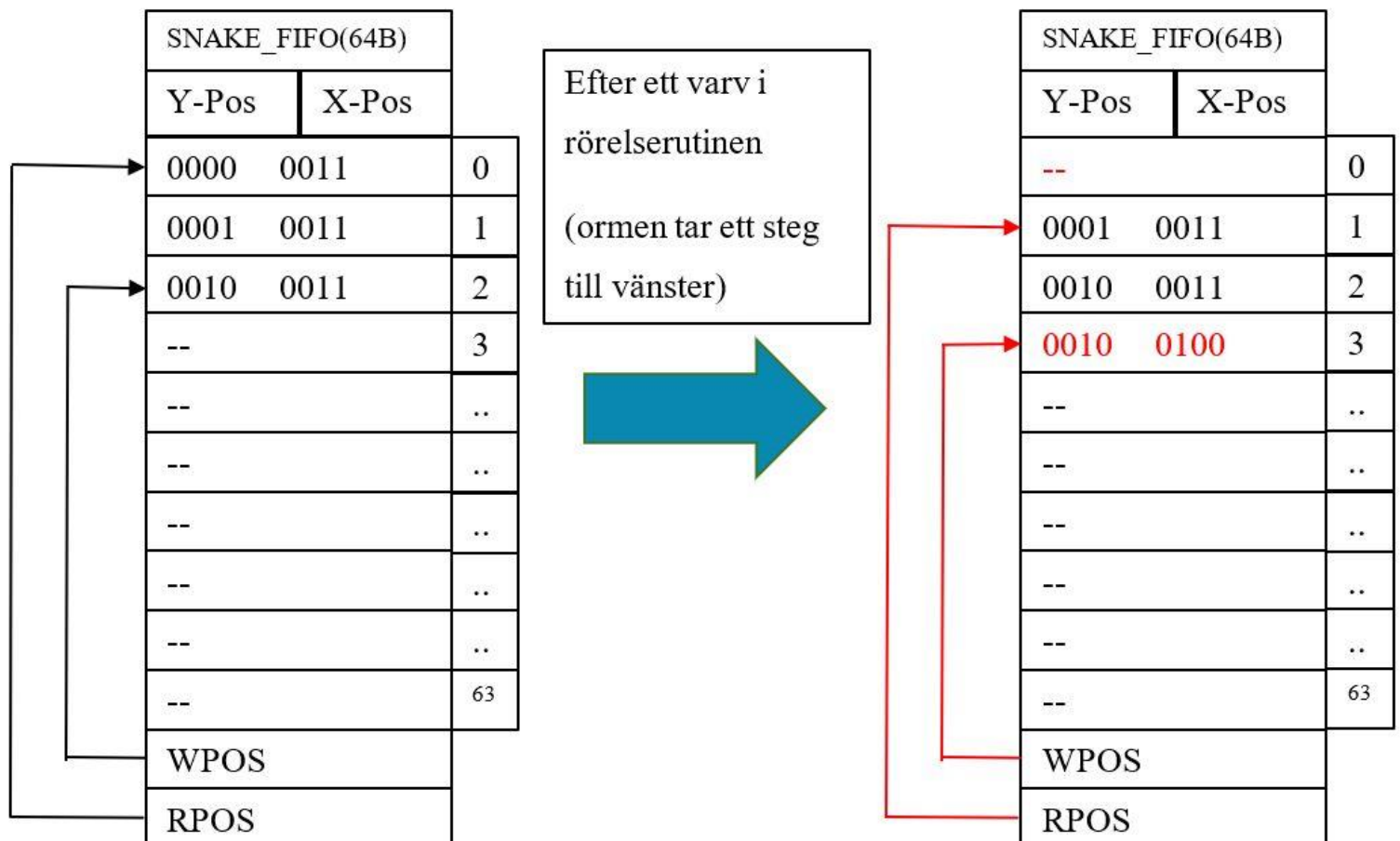
```
call CHECK_DIR
call FIFO_PUSH      ;IN(r20) NEW HEAD POS
call FIFO_POP       ;OUT(r20) HEAD POS
call SET_PIX        ;IN=r20, OUT=r19(X) r16(Y)
call UPDATE_SNAKE   ;SNAKE_FIFO -> SNAKE
call CHECK_HIT      ;CMP r20(head pos) with FOOD_POS
```

```
.org      0x300
SNAKE_FIFO: .byte 64
RPOS:      .byte 1
WPOS:      .byte 1
```

Figur 8: Minnes deklarationen

Figur 9: Rörelselooopen i FIFO:n

4.4.1 Videominnet (SNAKE_FIFO)



Figur 20: Illustrering av minneshantering.

Enligt figur 10, loopas minnet och positionerna rör sig likt en orm genom alla 64 Bytes, detta för att underlätta ormens rörelse i och med att ormens bakre del alltid tar koordinaterna från kroppen framför. För att navigera i detta evigt loopande minnet finns WPOS (Write-/Head Position) samt RPOS (Read-/Tail Position) till hjälp som kan användas för att spara och läsa på rätt plats i minnet, men också ha ett sätt att kunna räkna ut ormens längd. Detta gör dessutom att med hjälp av WPOS kan huvudets position avläsas när som helst, som bland annat används för att kontrollera om ormen har kolliderat med mat.

4.4.2 Översättning av koordinater (SET_PIX)

Från Snake FIFO kommer alltså koordinater som ej går att direkt applicera i muxen. därför krävs det någon slags översättning till muxens format. Detta är löst på två sätt, för att behandla Y-positionen tas de fyra högsta bitarna från FIFO:n och adderas med en pekare som pekar på videominnet i muxen vilket innebär att pekaren hamnar på rätt position, vilket är ekvivalent till muxens Y-position. X-positionen är löst på ett annorlunda sätt, där det egentligen X-värdet läses från de fyra minsta bitarna från FIFO:n och skiftar ett register (utgångsvärde 0000 0001) X antal gånger för att ettan som söks skall hamna på rätt position.

4.4.3 Push och Pop

Rutinerna i figur 11 är de verktyg som möjliggör hela minneshanteringen. Där push tar in nya positionen efter en rörelse skett i joysticken och sparar de på rätt plats i FIFO-minnet. Den kontrollerar även ifall WPOS nått slutet på de allokerade 64 Bytes:en och hoppar till början.

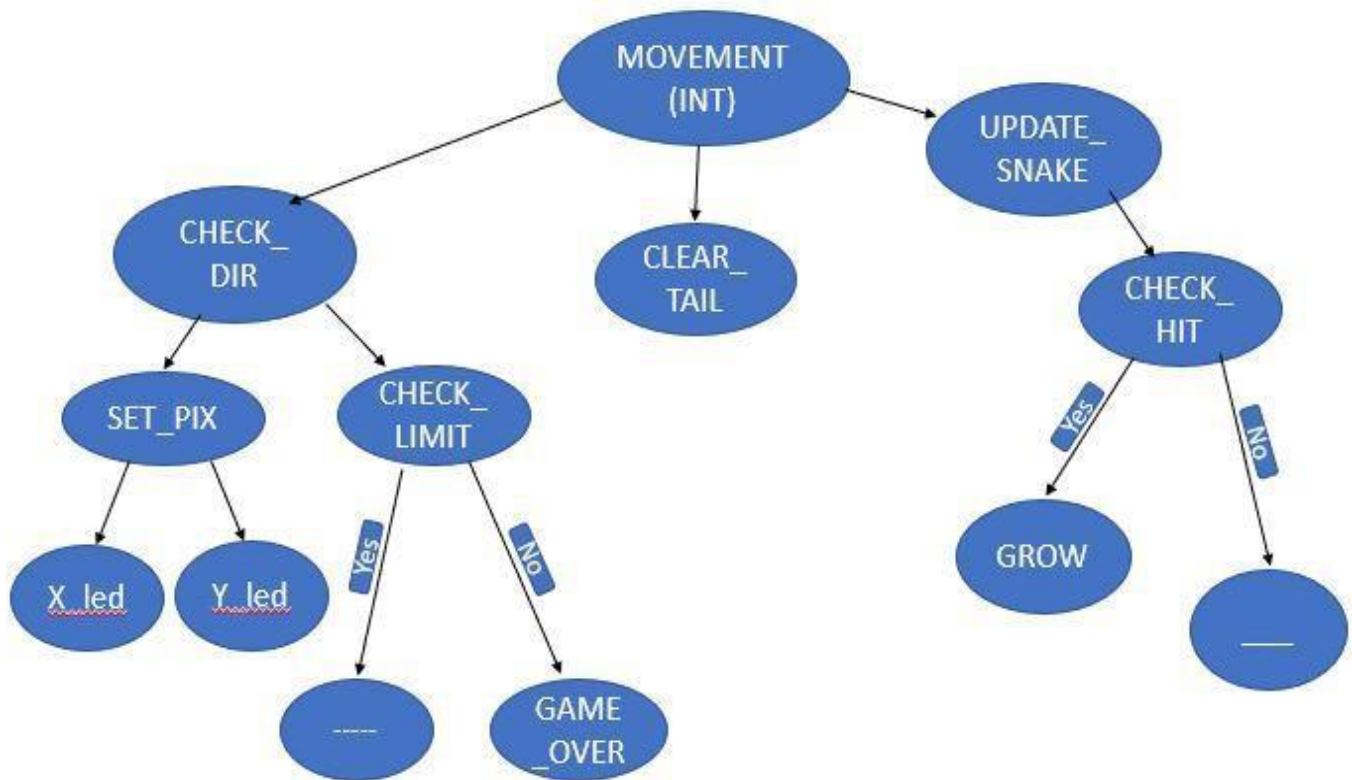
Pop-funktionen laddar huvudets position i ett register från minnet som därefter kan användas för att senare flytta ormen i rörelserutinen. Den kontrollerar och återställer även RPOS så den inte går utanför minnesutrymmet.

```
//IN r20=val ;;;SETS COORD VALUE
FIFO_PUSH:
    lds    r16, RPOS
    lds    r17, SNAKE_LEN
    add    r16, r17
    ldi    ZH, HIGH(SNAKE_FIFO)
    ldi    ZL, LOW(SNAKE_FIFO)
    inc    r16
    andi   r16, 63
    sts    WPOS, r16
    add    ZL, r16
    st     Z, r20
    ret

//OUT r20=val ;;;READS VALUE FROM
FIFO_POP:
    lds    r16, RPOS
    ldi    ZH, HIGH(SNAKE_FIFO)
    ldi    ZL, LOW(SNAKE_FIFO)
    inc    r16
    andi   r16, 63
    sts    RPOS, r16
    lds    r16, WPOS
    add    ZL, r16
    ld     r20, Z
```

Figur 11: Kodexempel på
FIFO_PUSH och FIFO_POP

4.4.4 FIFO JSP-Diagram

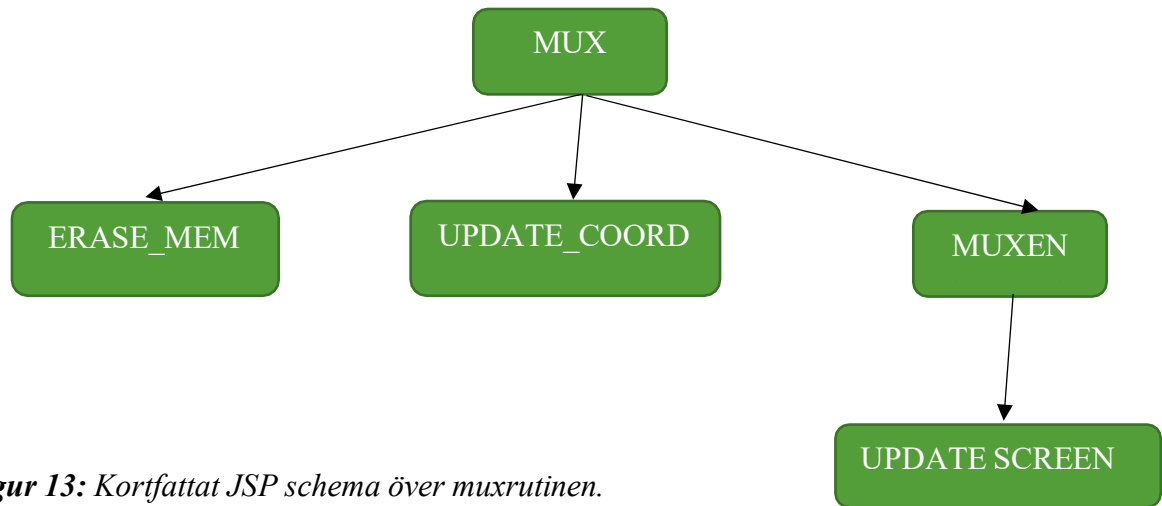


Figur 12: FIFO JSP-diagram.

4.5 Muxen

Muxen är en vital del av projektet då den hanterar uppdateringen av displayen. Muxens huvudsakliga uppgift är att hantera VMEM (Video Memory) för att lagra den information som ska skickas ut på skärmen, men även att skicka ut informationen till skärmen via SPI-protokollet nämnt tidigare.

Muxen anropas via en timer overflow interrupt, och fungerar genom först rensa nuvarande VMEM för att sedan uppdatera VMEM med de nya koordinaterna givna efter rörelse. Den nya informationen skickas sedan till skärmen en rad åt gången. Detta betyder att varje gång muxen anropas så uppdaterar den enbart en rad på Y-axeln, för att sedan till nästa anrop uppdatera nästa rad på Y-axeln.



Figur 13: Kortfattat JSP schema över muxrutinen.

```
ldi    ZH, HIGH(VMEM_B)
ldi    ZL, LOW(VMEM_B)
call   ERASE_MEM
call   UPDATE_COORD
```

Figur 14: Main rutinen för MUX.

```

MUXEN:
    lds    r17, RAD_INDEX
    ldi    ZH, HIGH(VMEM_B)
    ldi    ZL, LOW(VMEM_B)
    add    ZL, r17
    ld     r16, Z
    call   SPI

    ldi    ZH, HIGH(VMEM_G)Z
    ldi    ZL, LOW(VMEM_G)
    add    ZL, r17
    ld     r16, Z
    call   SPI

    ldi    ZH, HIGH(VMEM_R)
    ldi    ZL, LOW(VMEM_R)
    add    ZL, r17
    ld     r16, Z
    call   SPI

    ldi    ZH, HIGH(RAD*2)
    ldi    ZL, LOW(RAD*2)
    add    ZL, r17
    lpm    r16, Z
    call   SPI

UPDATE_COORD:
    lds    r16, SNAKE
    sts    VMEM_B, r16
    lds    r16, SNAKE+1
    sts    VMEM_B+1, r16
    lds    r16, SNAKE+2
    sts    VMEM_B+2, r16
    lds    r16, SNAKE+3
    sts    VMEM_B+3, r16
    lds    r16, SNAKE+4
    sts    VMEM_B+4, r16
    lds    r16, SNAKE+5
    sts    VMEM_B+5, r16
    lds    r16, SNAKE+6
    sts    VMEM_B+6, r16
    lds    r16, SNAKE+7
    sts    VMEM_B+7, r16

    lds    r16, FOOD
    sts    VMEM_G, r16
    lds    r16, FOOD+1
    sts    VMEM_G+1, r16
    lds    r16, FOOD+2
    sts    VMEM_G+2, r16
    lds    r16, FOOD+3
    sts    VMEM_G+3, r16
    lds    r16, FOOD+4
    sts    VMEM_G+4, r16
    lds    r16, FOOD+5
    sts    VMEM_G+5, r16
    lds    r16, FOOD+6
    sts    VMEM_G+6, r16
    lds    r16, FOOD+7
    sts    VMEM_G+7, r16

    ret

```

Figur 15: *Koden för att skicka VMEM till skärmen. RAD_INDEX incrementas mellan varje MUX anrop.*

Figur 16: *Rutinen för att uppdatera VMEM med ny information från till exempel rörelse rutinen.*

5 Diskussion

5.1 Misstag

1. Vi bestämde oss först att använda fyra 8x8 DAmatrix-displayer för en total storlek på 16x16. Dock insåg vi att det blev mycket svårt att sammankoppla mellan displayerna när det kom till uppdatering på ormens kropp.
2. I det blockschemat som är inlämnat till kunden visas två joystick som skulle användas, men under projektets gång kom vi fram till att det räckte med en joystick.
3. På grund av problemen som uppstod i försöket att använda fyra DAmatrix-displayer, så tog det längre tid innan vi lyckades få något mer än enstaka punkter på displayen att lysa upp. Detta medförde att vi började leta efter inspiration från andra grupper och tillgängliga assistenter, vilket ledde till många olika förslag. Detta var positivt men tyvärr så ledde det samtidigt till mycket förvirring.
4. Något vi bör ha agerat på direkt var att så fort vi tappat en gruppmedlem borde vi ha satt oss och planerat om hela projektet då tidsplanen blev omöjlig att följa med en man mindre.
5. Eftersom alla viktiga funktioner i projektet körs på interrupts, har vi inte kunnat skapa en JSP-diagram som kan täcka alla funktioner på ett effektivt sätt.
6. Dokumentationen fungerade tyvärr inte som vi hade planerat från början. Vi skrev och antecknade för litet.

5.2 Slutsats

Vi är i helhet nöjda med vårt projekt, men vi kunde ha varit bättre på att följa vår planering när det kom till anteckning och dokumentation. Vi hade också gärna velat lägga till mer funktionalitet och polerat den nuvarande men tyvärr sprang tiden i väg.

6 Figurförteckning

Figur 1: Blockschema.....	5
Figur 2: De I/O pinar som inkluderar de 24 pinnarna för Port B, Port C och Port D och även 4 andra enskilda I/O pinnar	6
Figur 3: SPI Master-Slave Interconnection	7
Figur 4: Kodexempel på pinndefinitioner och HW_INIT för start av SPI.....	8
Figur 5: Endast de två högsta bitarna från X respektive Y-led från joysticken är relevanta för projektet och används för att läsa av joystickens position. Spelaren kan enbart gå vågrätt eller lodrätt, vilket gör att det endast är de gråmarkerade rutorna i diagrammet	8
Figur 6: Kodutdrag för joystick funktionen. ADMUX sätts upp för att hantera Y-axel respektive X-axel, omvandling startas och joystickvärde söks. Vid hittat värde lagras loopcount värde.....	9
Figur 7: Initiering av interrupten för joysticken	9
Figur 8: Minnes deklarationen.....	10
Figur 9: Rörelselooopen i FIFO:n	10
Figur 10: Illustrering av minneshantering	10
Figur 11: Kodexempel på FIFO_PUSH och FIFO_POP	11
Figur 12: FIFO JSP-diagram	12
Figur 13: Kortfattat JSP schema över muxrutinen	13
Figur 14: Main rutinen för MUX.....	13
Figur 15: Koden för att skicka VMEM till skärmen. RAD_INDEX incrementas mellan varje MUX anrop	14
Figur 16: Rutinen för att uppdatera VMEM med ny information från till exempel rörelse rutinen	14

7 Referenslista

Data-blad ATmega328P.pdf

https://liuonline.sharepoint.com/:b:/r/sites/Lisam_TSIU51_2023VT_K5/Delade%20dokument/General/Kursdokument/Datablad_ATMega328.pdf?csf=1&web=1&e=bBKXZC

David_hardvarubeskrivning.pdf

https://liuonline.sharepoint.com/:b:/r/sites/Lisam_TSIU51_2023VT_K5/Delade%20dokument/General/Kursdokument/David_hardvarubeskrivning.pdf?csf=1&web=1&e=MpMMxZ

DAmatrix – 8x8 RGBLED.pdf

https://liuonline.sharepoint.com/:b:/r/sites/Lisam_TSIU51_2023VT_K5/Delade%20dokument/General/DAvid%20datasheets/DAmatrix%20-%208x8%20RGBLED.pdf?csf=1&web=1&e=0OT9vi

AVR-programmering.pdf

https://liuonline.sharepoint.com/:b:/r/sites/Lisam_TSIU51_2023VT_K5/Delade%20dokument/General/DAvid%20datasheets/DAmatrix%20-%208x8%20RGBLED.pdf?csf=1&web=1&e=0OT9vi

Omslagsbild (Snake på en gammal Nokia)

https://melmagazine.com/wp-content/uploads/2019/09/Nokia_Snake.jpg